An LLM-Based Multimodal Framework for Automated SQAG Compliance Verification of Software Requirement Specifications**(20pt)**

Aayush Anand[1], Sai Charan (Sci-C)[2*], A.Kalidasu (Sci-E)[3]

[1] Department Of Computer Science And Engineering, DIAT, Pune, India
[2] Directorate of Artificial Intelligence and Visualization, RCI, Hyderabad, India
[3] Directorate of Artificial Intelligence and Visualization, RCI, Hyderabad, India
1st aayushanand1411@gmail.com 2nd author email; 3rd author email
*Corresponding Author: author@gmail.com

Abstract— *Manual verification of Software Requirement Specifications (SRS) against Software Quality Assurance Group (SQAG) guidelines is a critical but time-consuming task in safety-critical and defence software development. The process is further complicated by inconsistent document formats, semantic variation in section structures, and the presence of diagrams that are difficult to interpret using traditional text-based methods.*
This paper presents a multimodal large language model (LLM) based framework for automated SQAG compliance verification of SRS documents. The proposed system integrates document preprocessing, PDF-to-Markdown transformation, multimodal image captioning, semantic section mapping, and prompt-driven reasoning to evaluate document compliance against SQAG checklist requirements. Extracted textual and visual information is normalized into a structured representation and analyzed using a prompt-guided LLM to generate compliance verdicts, concise explanations, and machine-readable outputs for scoring.
The framework further introduces a hierarchical weighted scoring mechanism that aggregates checklist evaluations to produce interpretable document-level compliance scores. The system is implemented as a web-based application supporting automated analysis, configurable templates, and human-in-the-loop validation.

Experimental evaluation on SQAG-reviewed SRS documents demonstrates that the proposed approach can effectively automate initial compliance screening while maintaining transparency and interpretability. The results highlight the potential of multimodal LLM systems to significantly reduce manual review effort in software quality assurance workflows.

Keywords—An LLM-Based Multimodal Framework for Automated SQAG Compliance Verification of Software Requirement Specifications

I. INTRODUCTION

Software Requirement Specifications (SRS) form the foundation of the software development lifecycle, particularly in safety-critical and defence-oriented systems where requirement clarity and correctness directly influence system reliability. In such environments, incomplete or ambiguous requirements can propagate through later development stages, leading to costly redesign, implementation errors, and potential operational risks. To reduce these risks, organizations employ structured verification guidelines such as those defined by Software Quality Assurance Groups (SQAG), which provide checklists to evaluate whether an SRS document satisfies key quality attributes including completeness, consistency, traceability, and verifiability.

Despite the importance of these guidelines, the verification of SRS documents is still largely performed manually. Reviewers must analyze lengthy documents, interpret variations in section structures, and evaluate individual requirements against multiple checklist items. In real-world scenarios, SRS documents often deviate from standardized templates and may include diagrams, architectural figures, tables, or state-transition representations that contain important information not explicitly described in text. These factors make the verification process time-consuming and increase the likelihood of inconsistent assessments among reviewers. Traditional automation approaches based on rule-based checks or keyword matching are limited in their ability to interpret contextual meaning or handle multimodal content present in technical documents.

Recent advances in Large Language Models (LLMs) have demonstrated strong capabilities in contextual reasoning, natural language understanding, and explanation generation. These models provide new opportunities

for intelligent document analysis by enabling systems to interpret requirements in context rather than relying solely on rigid pattern matching. However, many existing applications focus primarily on information extraction or document classification and do not directly address structured compliance verification aligned with engineering checklists. Furthermore, purely text-based approaches are insufficient for SRS documents where critical information may also be embedded within diagrams and other visual elements.

To address these challenges, this paper presents a multimodal framework for automated SQAG compliance verification of Software Requirement Specifications. The proposed approach integrates document preprocessing, PDF-to-Markdown transformation, multimodal image captioning, semantic section extraction, and prompt-driven reasoning using large language models. Extracted document content is mapped to SQAG checklist requirements and evaluated to generate structured compliance decisions along with concise explanations. In addition, the framework incorporates a hierarchical weighted scoring mechanism that aggregates checklist results to produce interpretable compliance scores for both individual evaluation criteria and the overall document.

The proposed system is implemented as a web-based application that supports automated analysis, configurable document templates, and human-in-the-loop validation. By combining multimodal document understanding with structured compliance evaluation, the framework aims to significantly reduce manual review effort while maintaining transparency and interpretability in software quality assurance workflows.

II. RELATED WORKS

A. Automated Document Analysis

Automated document analysis has been widely studied as a means of transforming unstructured or semi-structured documents into machine-interpretable representations. Early approaches relied primarily on optical character recognition (OCR), rule-based text processing, and natural language processing techniques to extract information from digital documents. With the advancement of deep learning and document intelligence, more recent research focuses on models capable of understanding complex layouts that include tables, figures, and hierarchical structures. Systems such as modern document parsing pipelines and structured extraction benchmarks have demonstrated improved performance in identifying document elements and preserving structural relationships within technical documents [1], [4]. Despite these advancements, many document analysis systems still focus mainly on textual extraction and often struggle to effectively interpret heterogeneous document formats and multimodal elements.

B. Automated Compliance Check

Automated compliance checking (ACC) has emerged as an important research direction in domains where technical documents must satisfy regulatory or engineering standards. Traditional compliance verification systems typically rely on predefined rules, template matching, or domain-specific knowledge bases to identify whether documents adhere to required guidelines. While these approaches can perform well for structured inputs, they often lack flexibility when dealing with natural language descriptions or documents with irregular structures. In the context of software engineering, standards such as IEEE 830 and ISO/IEC/IEEE 29148 define guidelines for writing and evaluating software requirement specifications [13], [14]. However, verifying compliance with these standards remains largely manual and time-consuming. Recent studies have begun exploring the use of artificial intelligence and machine learning methods to support compliance evaluation, but many of these approaches still depend heavily on structured inputs or handcrafted rule sets.

C. Multimodal LLMs for Technical Documents

Recent advances in large language models and multimodal artificial intelligence have significantly improved the capability of automated systems to understand complex documents. Transformer-based architectures have enabled models to capture contextual relationships within large text corpora, forming the foundation for modern language models such as BERT and GPT [6], [11]. Furthermore, multimodal models that integrate visual and textual representations have shown strong performance in tasks such as document understanding, visual question answering, and reasoning over diagrams and images [9], [10]. These models enable systems to interpret both textual descriptions and visual components of documents, which is particularly important for technical artifacts that frequently include diagrams, flowcharts, and architectural illustrations.

D. Research Gap

Although significant progress has been made in document intelligence, automated compliance checking, and multimodal language models, several limitations remain when applying these techniques to software engineering documentation. Existing compliance verification approaches often rely on rigid templates or rule-based systems that struggle to handle the variability and semantic richness of real-world SRS documents. Additionally, many LLM-based studies focus on information extraction or classification tasks rather than structured checklist-driven

verification aligned with engineering standards. Moreover, most existing approaches do not fully incorporate multimodal document elements such as diagrams or technical figures that may contain essential requirement information. These limitations highlight the need for a unified framework that integrates multimodal document understanding with structured checklist-based reasoning and interpretable scoring mechanisms for automated SQAG compliance verification.

III. PROPOSED FRAMEWORK

A. System Architecture

The proposed framework is designed as a modular pipeline that performs automated SQAG compliance verification of Software Requirement Specification (SRS) documents through multimodal document understanding and large language model reasoning. The system processes an input SRS document and progressively transforms it into a structured representation that can be evaluated against SQAG checklist requirements. The overall architecture is organized into three main phases: data ingestion and enrichment, structural standardization, and automated logic verification.

In the first phase, **Data Ingestion and Enrichment**, the input SRS document in PDF format undergoes preprocessing to remove repetitive layout artifacts such as headers and footers. This step improves downstream text extraction and reduces noise introduced by document formatting. The cleaned document is then converted into a structured Markdown representation using a document parsing engine. During this stage, embedded images and diagrams are extracted separately from the document. Since technical diagrams often contain important requirement information, each extracted image is processed using a vision-language model to generate descriptive captions. These captions are then reintegrated into the Markdown document at the corresponding locations, producing an enriched textual representation that combines both textual and visual information.

The second phase, **Structural Standardization**, focuses on transforming the enriched document into a normalized structure suitable for automated reasoning. Section headings and their associated content are first extracted from the Markdown document. However, real-world SRS documents frequently use inconsistent section names or structures. To address this challenge, the framework performs semantic section mapping using sentence embeddings, fuzzy string matching, and rule-based refinement techniques. This hybrid strategy aligns extracted sections with standardized SQAG categories even when exact header matches are not present. The output of this stage is a normalized JSON representation of the document where each section is mapped to the most relevant checklist category.

In the third phase, **Automated Logic Verification**, the normalized document structure is evaluated against SQAG checklist questions using prompt-driven reasoning with a large language model. For each checklist item, the system dynamically identifies relevant document sections and constructs a structured prompt that guides the model to perform compliance verification. The model produces a binary verdict indicating compliance or non-compliance along with a concise explanation grounded in the document content. Additionally, a structured JSON output is generated for each evaluation to support automated scoring and aggregation.

The outputs generated during the evaluation phase are subsequently used to compute weighted compliance scores based on the hierarchical scoring mechanism defined by the SQAG checklist. By combining multimodal document processing with structured LLM-based reasoning, the proposed architecture enables automated and explainable verification of SRS documents while maintaining transparency and traceability across the entire evaluation pipeline.

Fig. 1 illustrates the overall system architecture of the proposed framework and highlights the interactions between the document preprocessing pipeline, semantic section mapping module, and the prompt-driven compliance evaluation engine.

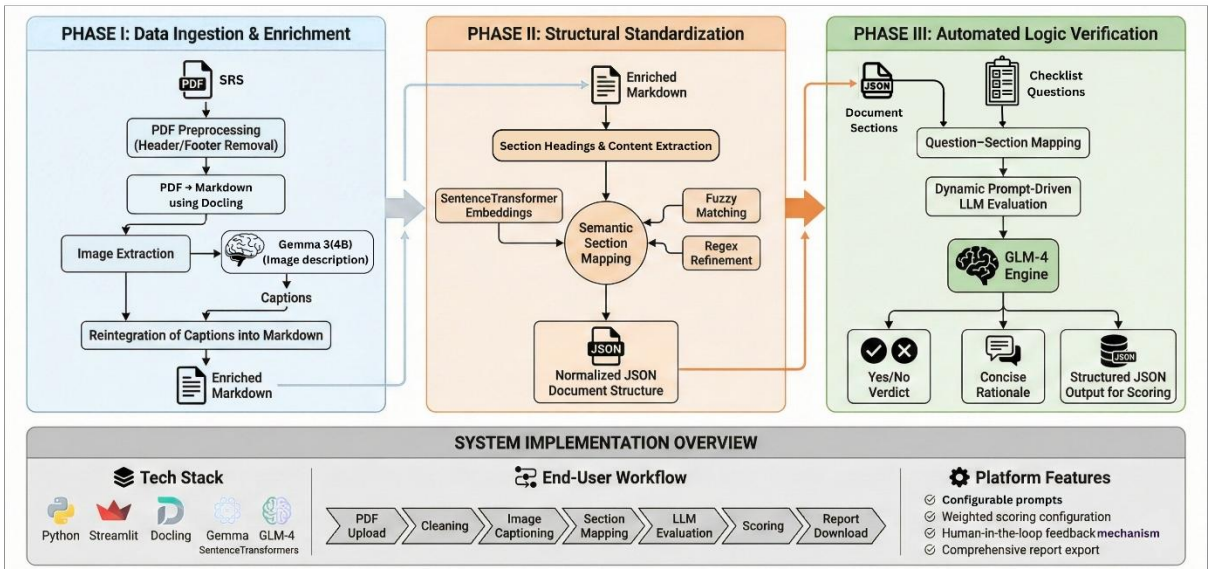

Fig. 1. System architecture of the proposed multimodal SQAG compliance verification framework.

B. Document Preprocessing and Markdown creation

The first step of the proposed framework focuses on converting raw Software Requirement Specification (SRS) documents into a structured representation suitable for automated analysis. In practice, SRS documents are commonly distributed in Portable Document Format (PDF), which often includes formatting artifacts such as page headers, footers, page numbers, and layout elements that are not relevant to requirement analysis. These artifacts can introduce noise during text extraction and negatively affect downstream processing. Therefore, an initial preprocessing step is applied to remove repetitive layout elements and retain only the meaningful textual content of the document.

After preprocessing, the cleaned PDF document is transformed into a Markdown representation using a document parsing engine. Markdown is selected as an intermediate format because it preserves the hierarchical structure of documents while remaining lightweight and easily processable. Important structural components such as headings, paragraphs, lists, and tables are maintained during this transformation, enabling the system to later identify logical sections within the document.

During the conversion process, the framework also detects and extracts embedded images and diagrams from the PDF file. These visual elements are temporarily separated from the textual content, and placeholders are inserted in the Markdown document at their original positions. This step preserves the contextual relationship between textual descriptions and visual elements within the document. The resulting Markdown file therefore maintains both the logical structure and positional information required for subsequent multimodal processing.

The output of this stage is a clean and structured Markdown document along with a collection of extracted images that are processed in the next stage for multimodal enrichment. This representation serves as the foundation for further analysis, including image captioning, semantic section extraction, and compliance evaluation.

C. Multimodal Image Captioning

Software Requirement Specification documents frequently include diagrams such as architectural representations, workflow diagrams, state-transition diagrams, and system interaction illustrations. These visual elements often contain important information that complements or extends the textual description of system requirements. Traditional document analysis pipelines typically ignore such visual content, which can lead to incomplete understanding of the document. To address this limitation, the proposed framework incorporates a multimodal image captioning module that converts visual information into textual descriptions that can be processed by downstream language models.

During the preprocessing stage, all images embedded within the PDF document are extracted and stored separately while maintaining their positional context within the Markdown document. Each extracted image is then processed using a vision-language model capable of generating descriptive captions for technical diagrams. The model is prompted to produce concise explanations describing the functional meaning of the diagram, such as system components, data flows, or state transitions represented in the image.

The generated captions are subsequently reintegrated into the Markdown document by replacing the image placeholders with their corresponding textual descriptions. This process enriches the document representation by combining visual semantics with textual content, enabling downstream components to analyze diagrams indirectly

through their generated descriptions. As a result, the system achieves a form of multimodal document understanding without requiring the language model to directly process image inputs during later reasoning stages.

D. Document Section Extraction and Semantic Mapping

After multimodal enrichment, the Markdown document is analyzed to extract its structural organization in terms of sections and subsections. Headings detected in the Markdown representation are used to segment the document into logical sections, where each section consists of a heading and the corresponding textual content that follows it. This section-wise segmentation allows the framework to process documents at a granular level rather than passing the entire document to the language model for every evaluation step.

However, real-world SRS documents often exhibit significant variation in section naming and ordering. For example, similar concepts may appear under headings such as “System Overview,” “General Description,” or “Functional Summary.” To ensure consistent interpretation, the framework performs semantic mapping between extracted document sections and predefined SQAG checklist categories.

The mapping process combines multiple techniques to improve robustness. Sentence-level embeddings are generated for both document section headings and standardized template headings using a sentence transformer model. Semantic similarity scores are computed to identify potential matches between document sections and checklist categories. These results are further refined using fuzzy string matching and rule-based regular expression filters to handle minor textual variations and formatting inconsistencies. Through this hybrid strategy, the framework aligns heterogeneous document structures with a consistent internal representation.

The output of this stage is a normalized JSON document structure in which each section is mapped to its corresponding SQAG category. This structured representation enables efficient retrieval of relevant sections during the compliance evaluation stage.

E. Prompt Driven Compliance Evaluation

The final stage of the framework performs automated compliance verification using prompt-driven reasoning with a large language model. Each SQAG checklist item represents a verification question that must be evaluated against the relevant portions of the SRS document. Instead of processing the entire document for every query, the system dynamically selects the most relevant document sections based on the mappings generated in the previous stage.

For each checklist question, a structured prompt is constructed that includes the selected document sections and the corresponding evaluation instruction. The prompt explicitly instructs the language model to determine whether the requirement described in the checklist item is satisfied within the provided document context. To ensure consistency and interpretability, the prompt also constrains the model to produce a structured output containing a binary compliance verdict (Yes or No) and a concise explanation grounded in the document content.

The language model processes the prompt and returns the evaluation result in a machine-readable format, typically structured as JSON. This structured output allows the system to automatically aggregate evaluation results and compute compliance scores using the hierarchical weighting mechanism defined in the SQAG checklist. By combining targeted context selection with structured prompting, the framework ensures that compliance decisions remain transparent, traceable, and aligned with the underlying document evidence.

F. Hierarchical Weighted Compliance Scoring

To provide an interpretable and importance-aware assessment of document quality, the proposed framework employs a hierarchical weighted scoring mechanism aligned with the structure of the SQAG checklist. The checklist consists of multiple **main questions**, each of which may contain several **sub-questions** that evaluate specific aspects of the Software Requirement Specification. Both levels are associated with importance weights that reflect their relative contribution to the overall compliance evaluation.

At the **sub-question level**, the large language model evaluates each checklist item using the prompt-driven evaluation process described earlier. The model produces a binary result indicating whether the requirement is satisfied within the document context. A compliant requirement is assigned a value of **1**, while a non-compliant requirement receives a value of **0**. Each sub-question is associated with a predefined weight that represents its importance relative to the corresponding main question. These weights are defined by SQAG guidelines and typically take values between **0 and 1**, where higher values indicate greater significance.

The score for each **main question** is computed by aggregating the weighted results of its sub-questions. Since the weights are not constrained to sum to one, the weighted results are normalized by the total weight assigned to the sub-questions of that main question. The normalized score is then scaled to produce a value between **0 and 10**, enabling consistent interpretation across different checklist categories. The score of a main question is therefore computed as:

$$Score_{\{main\}} = \left(\backslash frac \left\{ \sum_{\{i=1\}_i^{\{n\}w}} r_i \right\} \left\{ \sum_{\{i=1\}_i^{\{n\}w}} \right\} \right) \times 10$$

Where

w_i = weight assigned to the i-th sub-question,
r_i = binary evaluation result for the i-th sub-question *(0 or 1),*
n = total number of sub-questions under the main question.

After computing scores for all main questions, the framework calculates the **overall document compliance score** using the weights assigned to each main question. Each main question score is multiplied by its corresponding importance weight, and the results are normalized by the sum of all main question weights. This normalization ensures that the final score remains within a consistent range regardless of the number of checklist items. The document score is computed as:

$$Score_{\{document\}} = \backslash frac \left\{ \sum_{\{j=1\}_j^{\{m\}W}} \times Score_{\{main_j\}} \right\} \left\{ \sum_{\{j=1\}_j^{\{m\}W}} \right\}$$

Where

W_j = weight assigned to the j-th main question,
Score_main_j = compliance score of the j-th main question,
m = total number of main questions in the checklist.

This hierarchical scoring strategy ensures that more critical verification criteria contribute proportionally more to the final document evaluation while maintaining transparency in how individual checklist results affect the overall score. The resulting score, typically expressed on a **0–10 scale**, provides reviewers with both a detailed and high-level assessment of SRS compliance with SQAG guidelines.

IV. SYSTEM IMPLEMENTATION

A. Web Application Workflow

To enable practical deployment and interaction with the proposed compliance verification framework, the system is implemented as a web-based application that integrates the document processing pipeline, language model evaluation, and scoring mechanisms into a unified interface. The application allows users to upload Software Requirement Specification documents and obtain automated SQAG compliance assessments with minimal manual intervention.

The workflow begins with the user uploading an SRS document in PDF format through the web interface. Once the document is submitted, the system automatically triggers the preprocessing pipeline, which performs document cleaning and converts the PDF into a structured Markdown representation. During this stage, embedded diagrams are extracted and processed through the multimodal image captioning module to generate textual descriptions that are integrated back into the document representation.

After multimodal enrichment, the system performs semantic section extraction and mapping to align the document structure with SQAG checklist categories. The mapped sections are then passed to the prompt-driven evaluation engine, where the large language model analyzes the document content against checklist questions. The evaluation results are returned in a structured format containing compliance verdicts and concise explanations. These results are subsequently processed by the hierarchical weighted scoring module to compute compliance scores for each checklist item and the overall document.

The final results are presented to the user through the web interface, where the compliance verdicts, explanations, and computed scores can be inspected. The application also provides an option to export a comprehensive compliance report for documentation or review purposes. The complete workflow of the system implementation is illustrated in Fig. 1.

B. Template-Configurable Compliance System

A key design feature of the proposed framework is its template-configurable architecture, which allows the same verification pipeline to be applied to different types of software engineering documents. While the core processing stages—including preprocessing, multimodal enrichment, section extraction, and LLM-based evaluation—remain unchanged, the verification logic can be adapted to new document standards through configurable checklist templates.

Each document type is associated with a predefined checklist structure consisting of main questions, sub-questions, and corresponding importance weights. For example, different organizations may follow different documentation standards or templates when preparing SRS documents. To support these variations, the framework allows developers to define a checklist and weight configuration corresponding to a particular template.

When a new document type or template is introduced, the developer performs a one-time configuration process that involves defining the checklist questions and creating corresponding prompts for each verification item. These

prompts guide the language model in evaluating document content according to the specified compliance criteria. Once defined, the prompts and checklist configuration are stored within the system and can be reused for future evaluations.

In the deployed application, users simply select the **document type** (e.g., SRS, SDD, or other software artifacts) and the **template standard** (such as organization-specific templates) before uploading a document. The system then automatically loads the corresponding checklist configuration and performs compliance verification using the same processing pipeline. This modular design significantly improves scalability and allows the framework to support multiple documentation standards without requiring modifications to the core system.

C. Human-in-loop Verification

Although large language models provide strong capabilities for contextual reasoning and document analysis, automated verification in safety-critical domains still requires human oversight. To address this requirement, the system incorporates a human-in-the-loop verification mechanism that allows domain experts to review and validate the results generated by the automated pipeline.

Within the application interface, each checklist evaluation is presented together with the corresponding compliance verdict and explanation produced by the language model. Reviewers can inspect the reasoning behind each decision and verify whether the referenced document sections adequately support the verdict. If necessary, users may provide feedback or override specific results to ensure correctness.

This interactive review process enhances trust and accountability in the automated verification system. Rather than replacing human reviewers, the framework acts as a decision-support tool that performs the initial compliance screening while allowing experts to focus on complex or ambiguous cases. By combining automated analysis with expert validation, the system achieves a balance between efficiency and reliability in SQAG-based document verification workflows.

The complete workflow of the system implementation is illustrated in Fig. 1

V. EXPERIMENTAL SETUP

A. *DATASET AND GROUND TRUTH*

The proposed framework is evaluated using a curated collection of Software Requirement Specification (SRS) documents reviewed according to SQAG verification guidelines. These documents represent real-world software documentation scenarios and contain typical characteristics such as heterogeneous section structures, varying terminology, and embedded technical diagrams. Such variations provide a realistic evaluation environment for assessing the robustness of the proposed multimodal compliance verification system.

To establish a reliable benchmark, ground-truth labels are generated through manual verification performed using the SQAG checklist. Each checklist item is evaluated by domain reviewers to determine whether the corresponding requirement is satisfied in the document. These expert-reviewed decisions serve as the reference baseline for comparing the automated predictions generated by the proposed system. For confidentiality and security reasons, document identifiers and sensitive information are anonymized during the evaluation process.

B. *Evaluation Procedure*

Each SRS document is processed through the complete verification pipeline described in Section III. The process begins with document preprocessing and Markdown conversion, followed by multimodal enrichment through image captioning and semantic section mapping. The normalized document representation is then evaluated against SQAG checklist questions using prompt-driven reasoning with a large language model.

For every checklist sub-question, the system generates three outputs: a binary compliance verdict (Compliance and non-compliance), a concise explanation grounded in the document content, and a structured JSON representation used for automated scoring. The results from individual sub-questions are aggregated using the hierarchical weighted scoring mechanism to compute scores at the main-question level and an overall document compliance score.

C. Evaluation Metrics

To assess the effectiveness of the proposed framework, both quantitative and qualitative evaluation metrics are considered. The binary compliance predictions produced by the system are compared with the ground-truth labels to compute standard classification metrics, including **accuracy, precision, recall, and F1-score**. These metrics provide insight into the system's ability to correctly identify both compliant and non-compliant requirements.

In addition to classification accuracy, the quality of the explanations generated by the language model is evaluated using qualitative metrics commonly used in large language model research. These include **faithfulness**, which measures whether the explanation is grounded in the document content; **relevance**, which evaluates the

alignment between referenced document sections and the checklist question; and **contextual accuracy**, which assesses whether the model correctly interprets the requirement within the appropriate technical context.

The combination of classification metrics and explanation-centered evaluation provides a comprehensive assessment of the proposed system's performance, ensuring that the framework is not only accurate in its compliance predictions but also transparent and interpretable for practical deployment in software quality assurance workflows.

VI. Result and Discussion

A. Document-Level Compliance Assessment

The proposed framework generates an overall compliance score for each evaluated SRS document based on the hierarchical weighted scoring mechanism described in Section III. As illustrated in Fig. 2, a representative SRS document evaluated by the system achieved an overall compliance score of **7.55 out of 10**. This score indicates that the majority of SQAG checklist requirements were satisfied while highlighting specific areas requiring further improvement.

Qualitative analysis of the evaluated document shows that it contained clearly defined system requirements, unique requirement identifiers, and minimal redundancy or contradictions. These characteristics contributed positively to the final score and demonstrate the capability of the system to identify well-structured SRS documents. By aggregating individual checklist results into a normalized score, the framework provides reviewers with a concise summary of document quality while still preserving traceability to the underlying verification decisions. The result interface generated by the application, which displays checklist evaluations and explanations, is shown in Fig. 2.

Document: [redacted] **SRS.pdf**

Final Score: 7.55/10.00

Document Status: PASS

Sr No.	Question	Sr no.	Sub-Question	Answer	Explanation
4	Is each requirement numbered or uniquely identifiable from other requirements? **Score: 10.00/10.00**	1	Is each requirement uniquely numbered?	Yes	All requirements are uniquely numbered as per the template.
		2	Does every Unique ID cover only one requirement?	Yes	Each unique ID describes only one requirement, with clear separation of tasks and functionalities.
		3	Are requirements redundant	No	There is no evidence of redundant requirements. Each requirement has a unique ID and description, with no repetition found.
5	Is each requirement written in clear, concise, unambiguous language ? **Score: 10.00/10.00**	1	Is the English language used to convey each requirement clear, unambiguous, and grammatically correct?	Yes	All requirements are written clearly in proper English, free from grammatical errors, and with no ambiguity.
		2	Does any requirement contradict another requirement directly or indirectly?	No	The document does not contain any direct or indirect contradictions between the requirements.

Download table (CSV) Save PDF

Fig. 2. Example result interface showing checklist evaluation and overall document compliance score.

B. Checklist-Level Classification Performance

To evaluate the classification capability of the system, the automated compliance predictions were compared with the ground-truth labels obtained from manual SQAG verification. The resulting classification behavior is summarized using a confusion matrix, shown in Fig. 3. According to the evaluation results, the system correctly identified **120 compliant cases** and **66 non-compliant cases**, while **15 false positives** and **14 false negatives** were observed.

These results demonstrate that the proposed system performs well in detecting compliant requirements while maintaining a balanced performance in identifying non-compliant cases. The relatively low number of false

positives indicates that the system rarely marks non-compliant requirements as compliant, which is particularly important in safety-critical verification workflows. Misclassifications primarily occur when relevant information is distributed across multiple sections of the document or when requirement descriptions are ambiguous.

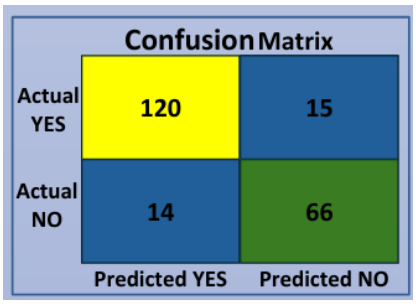

Fig. 3. Confusion matrix comparing automated compliance predictions with ground-truth SQAG verification.

C. Explanation Quality Evaluation

Beyond classification accuracy, the interpretability of model-generated explanations is an important factor for practical deployment in software quality assurance processes. To evaluate explanation quality, three qualitative metrics were considered: **faithfulness**, **relevance**, and **contextual accuracy**.

The evaluation results show that the system achieved a **faithfulness score of 0.783**, indicating that most explanations produced by the model were grounded in the corresponding document content. The **relevance score of 0.849** suggests that the system consistently references appropriate sections of the document when generating explanations for compliance decisions. The **contextual accuracy score of 0.616** reflects the system's ability to correctly interpret requirements within their intended technical context.

These results indicate that while the system demonstrates strong grounding and relevance, further improvements may be possible in capturing complex contextual relationships between requirements and diagrams. The evaluation metrics derived from the confusion matrix and explanation analysis are summarized in Fig. 4.

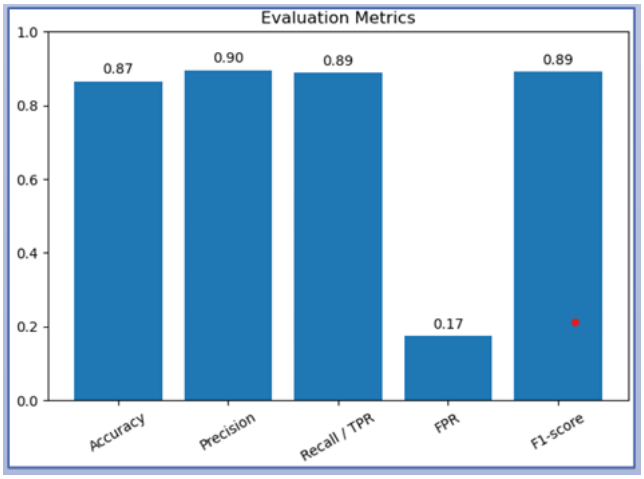

Fig. 4. Explanation quality metrics derived from the evaluation results.

VII. CHALLENGES AND LIMITATIONS

While the proposed framework demonstrates promising results in automating SQAG compliance verification of Software Requirement Specification documents, several challenges remain. One of the primary challenges arises from the variability in real-world SRS formats. In practice, documents often deviate from standardized templates, with sections merged, renamed, or organized differently. Although the semantic section mapping approach mitigates this issue to a large extent, extreme variations in document structure may still lead to partial misalignment between document content and checklist categories.

Another limitation involves the interpretation of complex diagrams and figures embedded within technical documents. While the multimodal image captioning module converts visual information into textual descriptions, certain technical nuances—such as implicit relationships between system components or detailed behavioral logic—may not be fully captured by automatically generated captions. As a result, some contextual information encoded in diagrams may not be completely reflected during compliance evaluation.

Semantic variation in requirement descriptions also presents challenges. Different authors may express similar requirements using diverse terminology or levels of abstraction. Although embedding-based similarity and fuzzy matching techniques help address this issue, subtle semantic differences can still affect section mapping and downstream reasoning. Additionally, the performance of the system is influenced by prompt design and grounding constraints within the language model. Variations in prompt phrasing or context selection may impact the consistency of evaluation results.

Finally, as with most LLM-based systems, the proposed framework does not provide formal guarantees of correctness. The automated decisions generated by the model should therefore be considered as decision-support outputs rather than definitive compliance judgments. Human oversight remains essential, particularly in safety-critical defence environments where verification accuracy is paramount.

VIII. CONCLUSION

This paper presented a multimodal large language model–based framework for automated SQAG compliance verification of Software Requirement Specification documents. The proposed system integrates document preprocessing, multimodal enrichment, semantic section mapping, and prompt-driven reasoning into a unified pipeline capable of analyzing complex technical documentation. By combining textual and visual information with structured checklist evaluation, the framework enables automated and explainable verification of SRS documents.

Experimental evaluation demonstrates that the system can effectively identify compliance with SQAG checklist requirements while providing interpretable explanations and weighted scoring for each evaluation criterion. The hierarchical scoring mechanism further enables reviewers to obtain both detailed insights and an overall assessment of document quality. Through the integration of multimodal document processing and large language model reasoning, the framework significantly reduces manual review effort while maintaining transparency in the verification process.

The proposed system is designed as a decision-support tool rather than a replacement for expert reviewers. By incorporating human-in-the-loop validation, the framework ensures that automated assessments remain interpretable and verifiable within real-world quality assurance workflows. Overall, the results highlight the potential of multimodal LLM-based systems to support intelligent document verification in defense-oriented software engineering environments and other domains requiring structured compliance analysis.

ACKNOWLEDGEMENT

If any funding or institutional acknowledgments are required, please include them before the References section using the appropriate format.

Note: Do **not** place acknowledgments in the footer of the document.

REFERENCES

[1] B. Pfitzmann, J. Ansel, N. Lysak, et al., “Docling v2: Unified document parsing and conversion pipeline,” *arXiv preprint arXiv:2501.17887*, 2025.

[2] Z. Li, A. Abulaiti, Y. Lu, X. Chen, J. Zheng, H. Lin, X. Han, S. Jiang, B. Dong, and L. Sun, “READoc: A unified benchmark for realistic document structured extraction,” *Findings of the Association for Computational Linguistics (ACL)*, 2025.

[3] T. Brown, B. Mann, N. Ryder, et al., “Language models are few-shot learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.

[4] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention is all you need,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.

[6] A. Radford, J. W. Kim, C. Hallacy, et al., “Learning transferable visual models from natural language supervision,” *Proceedings of the International Conference on Machine Learning (ICML)*, 2021.

[7] OpenAI, “GPT-4 Technical Report,” *arXiv preprint arXiv:2303.08774*, 2023.

[8] IEEE Computer Society, “IEEE recommended practice for software requirements specifications,” *IEEE Std 830-1998*, 1998.

[9] ISO/IEC/IEEE, “Systems and software engineering — Life cycle processes — Requirements engineering,” *ISO/IEC/IEEE 29148:2018*, 2018.